Curs 4 Programarea Calculatoarelor 2

Recursivitate (directă) = proprietatea unei funcții de a se autoapela

$$n! = \underbrace{1 \cdot 2 \cdot 3 \cdot \ldots \cdot (n-1)}_{(n-1)!} \cdot n = n\,(n-1)!$$

$$\begin{cases} n\,(n-1)! & , \text{dc } n \geq 1 \\ 1 & , \text{dc } n = 0 \end{cases}$$

$4! = 4 \cdot 3! = 4 \cdot 6 = 24$

$3! = 3 \cdot 2! = 3 \cdot 2 = 6$

$2! = 2 \cdot 1! = 2 \cdot 1 = 2$

$1! = 1 \cdot 0! = 1 \cdot 1 = 1$

$0! = \underline{\underline{1}}$

Contextul de apel al unei funcții

- numele funcției (1)
- valorile parametrilor (2) transmiși prin valoare (copii)
- variabilele locale (3)
- valoarea returnată (4)
- adresa de revenire din apel (5)

Un apel al unei funcții implică un context de apel salvat pe stivă (STACK)

```
int f (int n)
{
    if (n==0)
        return 1;
    return n * f(n-1);
}
- - - -
a = f(4); - - - - - - - - →
- - - -
```

A5 A4 A3 A2 A1

f	0	–	1	A5
f	1	–	1	A4
f	2	–	2	A3
f	3	–	6	A2
f	4	–	24	A1
1	2	3	4	5
	S	T	A C	K

4 nu e definit la început

A_1, A_2, A_3 sunt adrese din zona de cod

<u>Complexitatea unei funcții recursive</u> = nr. autoapelări

1 iterație ⇔ 1 apel

<u>Ex:</u>

1 Suma cifrelor unui nr nat.

```
int sumacif (int n){
    if (n < 10)
        return n;
    return n%10 * sumacif(n/10);
}
```

$O(\log_{10} n)$;

Pp că n are k cifre $\Rightarrow$

$\Rightarrow 10^{k-1} \le n < 10^k \Rightarrow$

$\Rightarrow k-1 \le \log_{10} n < k \Rightarrow$

$\Rightarrow [\log_{10} n] = k-1 \Rightarrow k = 1 + [\log_{10} n]$

② Alg lui Euclid pt cmmdc

$a = 180 \quad b = 14$

$\overset{a}{180} : \overset{b}{14} = 12$, rest 12

$14 : 12 = 1$, rest 2

$12 : 2 = 6$, rest 0

cmmdc $(180, 14) = 2$

$O(\log_2(\min(a, b)))$

cmmdc $(0,0) = m$ e def

cmmdc $(x, 0) = x$

$a = 14$, $b = 180$

Varianta iterativă

```
int r;          pp a, b ≠ 0
while(a){
  r = a % b
  a = b
  b = r
}
return a;
```

Varianta recursivă

$$cmmdc(a,b) = \begin{cases} cmmdc(b, a \% b), & \text{dc } b \neq 0 \\ a, & \text{dc } b = 0 \end{cases}$$

③ Șirul lui Fibonacci (Șirul lui Leonardo)

$$\begin{cases} f_0 = 0 \\ f_1 = 1 \\ f_n = f_{n-2} + f_{n-1} \end{cases}$$

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Varianta recursivă $O(2^n)$

$\boxed{\overset{fa}{0}, \overset{fb}{1}, 1}$, 2, 3, 5, 8, 13, 21, 34, ...

$c = a + b$

Sliding window iterative

Varianta iterativă: $O(n)$

④ Suma elementelor dintr-un tablou unidim

$$suma(v, n) = \begin{cases} v[n] + suma(v, n-1), & \text{dc } n \geq 2 \\ v[0], & \text{dc } n = 1 \end{cases}$$

⑤ Suma elementelor strict pozitive

```
int sumpar (int v[], int n)
{   if (n==0) return 0;
    if (v[n-1] > 0) return v[n-1]+sumpar(v, n-1)
    return sumpar (v, n-1);
```

6) din 4 în $n \in \mathbb{N}$

1) se împarte ~~[illegible]~~ pe cifra la 2

2) se adaugă cifra 4 la sfârșitul numărului

3) se adaugă cifra 0 la sfârșitul numărului

Ex: $n = 101 \Rightarrow 4 \xrightarrow{3)} 40$ $\quad 202 \xrightarrow{1)} 101$

$40 \xrightarrow{2)} 404$

$404 \xrightarrow{(1)} 202$

~~[illegible]~~

1) se înmulțește cu 2

2) dc ult. cifră este 4, at. se elimină

3) dc ult cifră este 0, at. se elimină

$n = 101 \xrightarrow{1)} 202 \xrightarrow{1)} 404 \xrightarrow{2)} 40 \xrightarrow{3)} 4$